

The Claude Beast

Claude Hooks

Deterministic Control Over Probabilistic AI

Episode 04 | The Claude Beast Series

February 2026

What Are Claude Code Hooks?

User-defined shell commands, LLM prompts, or subagents that execute automatically at specific points in Claude Code's lifecycle.

- ▶ **Not suggestions** — guaranteed executions that fire every time conditions are met
- ▶ Run with the user's full system permissions (no sandbox)
- ▶ Communicate via stdin (JSON), stdout (JSON/text), stderr, and exit codes
- ▶ Analogous to Git hooks (pre-commit, post-merge) but for AI development
- ▶ **Originally 7 events (June 2025)** — now 14+ lifecycle events (Feb 2026)



“Instead of hoping Claude remembers to lint your files, a `PostToolUse` hook runs Prettier automatically, every single time.”

14+ Lifecycle Events



Session

- SessionStart
- SessionEnd
- PreCompact



Tool

- PreToolUse
- PostToolUse
- PostToolUseFailure
- PermissionRequest



Agent

- Stop
- SubagentStart
- SubagentStop
- UserPromptSubmit



New (2025-26)

- TeammateIdle
- TaskCompleted
- ConfigChange
- WorktreeCreate/Remove

Three Hook Types

Command

"type": "command"

Run a shell script or command

- ▶ Fast execution (milliseconds)
- ▶ JSON in via stdin, exit codes out
- ▶ Best for: formatting, linting, security checks, logging

```
jq -r '.tool_input.file_path' | xargs prettier --write
```

Prompt

"type": "prompt"

Ask an LLM a yes/no question

- ▶ Single-turn LLM decision
- ▶ Returns {ok: true/false, reason}
- ▶ Best for: complex decisions, semantic validation

```
"Check if all tasks are complete. Respond with {ok: false} if incomplete."
```

Agent

"type": "agent"

Spawn a multi-turn subagent

- ▶ Up to 50 tool turns
- ▶ Access to Read, Grep, Glob, Bash
- ▶ Best for: multi-file checks, test suites, deployments

```
"Verify all unit tests pass. Run the suite and check results. $ARGUMENTS"
```

Where Hooks Live



Location	Scope	Description	Shareable?
<code>~/.claude/settings.json</code>	User-wide	All your projects	No
<code>.claude/settings.json</code>	Project	Single project (shared)	Yes – commit to repo
<code>.claude/settings.local.json</code>	Project (private)	Single project (personal)	No – gitignored
Managed policy settings	Organization	Admin-controlled	Yes – central policy
Plugin hooks/hooks.json	Plugin	When plugin is enabled	Yes – bundled
Skill / agent frontmatter	Component	While skill is active	Yes – in component



Structure

```
{ "hooks": { "PreToolUse": [{ "matcher": "Bash", "hooks": [{ "type": "command", "command": "/path/to/script.sh" }]}] } }
```

Matcher Patterns: Surgical Precision

Pattern Syntax

"Bash"	Exact tool name match
"Edit Write"	Pipe-separated alternatives
"*" or ""	Wildcard – match everything
"Bash(npm test*)"	Tool + argument pattern
"mcp__github__.*"	Regex for MCP tools
"mcp__.*__write.*"	Cross-server MCP regex

Event-Specific Filters

PreToolUse / PostToolUse	Tool name
SessionStart	startup, resume, compact, clear
SessionEnd	clear, logout, other
Notification	permission_prompt, idle_prompt
SubagentStart / Stop	Agent type
ConfigChange	Config source
UserPromptSubmit / Stop	No matcher (always fires)

⚠️ Matchers are case-sensitive: "bash" will NOT match the Bash tool. Use PascalCase.

Guidance vs. Enforcement



CLAUDE.md = Suggestions

Probabilistic

- ▶ Behavioral guidance interpreted by the LLM
- ▶ Can be forgotten, deprioritized, or misinterpreted
- ▶ May be lost after context compaction
- ▶ Best for: preferences, coding style, conventions



Hooks = Enforcement

Deterministic

- ▶ Guaranteed executions that run every time
- ▶ Independent of model cooperation
- ▶ Cannot be forgotten or deprioritized
- ▶ Best for: hard rules, security, mandatory formatting



Decision Framework

“Prefer Bun over npm”

→ **CLAUDE.md**

A preference, not a hard rule

“Always format files on save”

→ **Hook (PostToolUse)**

Must happen every time

“Never modify .env files”

→ **Hook (PreToolUse)**

Hard block, not a suggestion

“Verify tests pass before stopping”

→ **Hook (Stop)**

Enforcement gate

Real-World Examples



Auto-Format Code

PostToolUse | Write|Edit

Run Prettier on every file
Claude writes or edits.
Always exit 0 so it never
blocks Claude accidentally.

```
jq -r '.tool_input.file_path'  
| xargs prettier --write  
2>/dev/null; exit 0
```



Block Dangerous Commands

PreToolUse | Bash

Prevent destructive commands
before they execute: rm -rf,
DROP TABLE, --force push.
Exit code 2 = blocked.

```
# Check for dangerous patterns  
for p in "rm -rf /" "DROP TABLE"  
"--force"; do  
  grep -qiE "$p" && exit 2  
done; exit 0
```



Auto-Test on Stop

Stop | (always fires)

Prevent Claude from stopping
until all tests pass. Check
stop_hook_active to avoid
infinite loops.

```
# CRITICAL: prevent loops  
if stop_hook_active; then  
  exit 0  
fi  
npm test || exit 2
```

Async Hooks & Advanced Patterns



Async Hooks

`"async": true`

- ▶ Run background tasks without blocking the workflow
- ▶ Background test execution after file changes
- ▶ Audit logging that won't slow down development
- ▶ Documentation generation triggered by code changes
- ▶ Desktop notification dispatch



Advanced Patterns

- ▶ Git branch-per-session (GitButler pattern)
- ▶ Auto-approve safe operations (PermissionRequest)
- ▶ Re-inject context after compaction (SessionStart)
- ▶ Configuration change auditing (ConfigChange)
- ▶ Desktop notifications on idle (Notification)



Execution Behaviors

- ▶ Multiple matching hooks run in parallel – identical commands are auto-deduplicated
- ▶ Default timeout: 10 minutes (600s) – configurable per hook via the timeout field
- ▶ Command hooks add milliseconds of overhead – prompt/agent hooks benefit from async
- ▶ Environment variables: CLAUDE_PROJECT_DIR, CLAUDE_ENV_FILE, CLAUDE_TOOL_INPUT_FILE_PATH

Key Takeaways



- 1.** Hooks are the killer extensibility feature that makes Claude Code uniquely programmable
- 2.** Deterministic enforcement beats probabilistic suggestions – use hooks for hard rules
- 3.** Three hook types (command / prompt / agent) serve speed, judgment, and verification needs
- 4.** 14+ lifecycle events cover the full session from start to finish
- 5.** Matchers provide surgical precision with regex support for tools and MCP servers
- 6.** Exit code 2 is the universal blocker – feed stderr back to Claude as feedback
- 7.** Async hooks enable background operations without blocking the workflow
- 8.** Security is your responsibility – hooks run with full user permissions, no sandbox